

Unidad N° 5: Conceptos intermedios de Laravel

Autenticación

La **autenticación** es el proceso técnico mediante el cual un sistema informático verifica la identidad de un usuario, dispositivo u otra entidad para confirmar que es quien dice ser. Esto se realiza para poder acceder a recursos de un sistema que están protegidos.

Dependiendo del sistema, se pueden seguir una serie de pasos, como ser:

1. **Identificación:** El usuario proporciona sus credenciales, las cuales como mínimo deben estar compuestas por un identificador único (nombre de usuario, correo electrónico u otro código que cumpla la condición de ser único para cada usuario) y una clave o contraseña.
2. **Verificación de Credenciales:** El sistema recibe las credenciales y las verifica para establecer si la identidad del usuario es correcta. Además, se pueden realizar comprobaciones adicionales en caso de ser necesario; por ejemplo: si el usuario actual se encuentra deshabilitado, no podrá iniciar sesión en el sistema.
3. **Validación y creación de la sesión:** Si las credenciales son correctas, se procede a crear una sesión de usuario en el sistema y redirigir al usuario a una página definida (un panel de administración, la página con los datos de su cuenta, u otra).

Sesión

Una **sesión web** es un mecanismo utilizado para mantener el estado de un usuario a lo largo de múltiples peticiones HTTP. Dado que el protocolo HTTP es **sin estado**, lo que significa que cada petición es independiente de las anteriores, la sesión permite que el servidor "recuerde" a un usuario y sus interacciones previas.

El proceso de una sesión web se basa en la interacción entre el servidor y el navegador del cliente:

1. **Creación de la Sesión:** Cuando un usuario interactúa por primera vez con una aplicación web, el servidor genera un **identificador único de sesión** (Session ID).
2. **Envío de la Cookie:** Este Session ID se almacena en una **cookie** que el servidor envía al navegador del usuario. Esta cookie no contiene datos sensibles, solo el identificador.
3. **Almacenamiento de Datos:** El servidor guarda los datos de la sesión (como el nombre de usuario, el estado de autenticación, o el contenido del carrito de compras) en el lado del servidor, asociándolos con el Session ID.
4. **Peticiones Subsecuentes:** En cada petición posterior, el navegador envía la cookie con el Session ID de vuelta al servidor.

5. **Recuperación de Datos:** El servidor utiliza el Session ID para buscar los datos de la sesión correspondientes en su almacenamiento, permitiendo a la aplicación acceder a la información del usuario y mantener la continuidad de la experiencia.

Ejemplo con Laravel: Sistema de autenticación

Para crear un sistema de autenticación desde cero, aplicaremos los siguientes cambios.

1) Definición de las rutas necesarias

Vamos a definir 4 rutas en nuestro sistema que cumplirán las siguientes funciones:

- Mostrar la página con el formulario de login
- Recibir los datos del formulario y validar las credenciales
- Mostrar la página “dashboard”, al cual se redirige a los usuarios que han iniciado sesión
- Cerrar la sesión cuando el usuario hace click en el respectivo botón

```
// Ruta para mostrar el formulario de login
```

```
Route::get('/login', [AuthController::class, 'showLogin'])
    ->name('login.show');
```

```
// Ruta para procesar el intento de login
```

```
Route::post('/login', [AuthController::class, 'storeLogin'])
    ->name('login.store');
```

```
// Ruta del dashboard para redirigir usuarios logueados
```

```
Route::get('/dashboard', function () {
    return '¡Bienvenido! Has iniciado sesión correctamente.';
})->name('dashboard');
```

```
// Ruta para cerrar sesión
```

```
Route::post('/logout', [AuthController::class, 'logout'])
    ->name('logout');
```

2) Crear el controlador AuthController

Usamos el comando para crear el controlador:

```
php artisan make:controller AuthController
```

Y luego agregamos los 3 métodos necesarios:

```
public function showLogin()
{
    return view('sesiones.login');
}

public function storeLogin(Request $request)
{
    $credentials = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required'],
    ]);

    if (Auth::attempt($credentials)) {
        // Si las credenciales son correctas, regenera la sesión
        para evitar ataques de fijación de sesión
        $request->session()->regenerate();

        // Redirige al usuario a la URL que intentó visitar antes
        de ser redirigido al login, o al 'home' por defecto
        return redirect()->intended('/dashboard');
    }

    // Si la autenticación falla, redirige de nuevo al formulario
    de login con un error
    return back()->withErrors([
        'email' => 'Las credenciales proporcionadas no coinciden
        con nuestros registros.',
    ])->onlyInput('email');
}

public function logout(Request $request)
{

```

```

Auth::logout(); // Cierra la sesión

$request->session()->invalidate(); // Invalida la sesión
$request->session()->regenerateToken(); // Regenera el token
CSRF

return redirect('/'); // Redirige al inicio
}

```

3) Crear la vista de la página login

Por conveniencia, agruparemos todas las vistas relativas a la sesión en la carpeta **resources/views/sesiones** (más adelante aquí podemos ubicar las vistas de registro y de recuperar contraseña, entre otras), y dentro de la misma crearemos el archivo **login.blade.php** con el siguiente contenido:

```

<form method="POST" action="{{ route('login.store') }}">
    @csrf
    <div>
        <label for="email">Email</label>
        <input id="email" type="email" name="email" required
autofocus>
    </div>

    <div>
        <label for="password">Contraseña</label>
        <input id="password" type="password" name="password"
required>
    </div>

    <div>
        <button type="submit">Iniciar Sesión</button>
    </div>
</form>

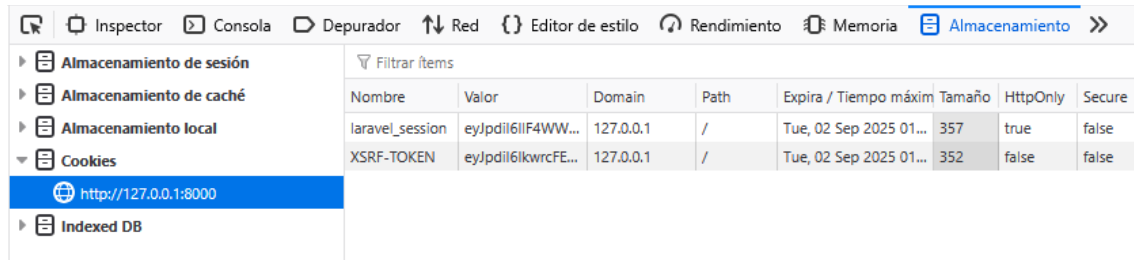
```

Nota que al ser un formulario se requiere la etiqueta `@csrf`; además, puedes vincular el resto de la vista con tu plantilla o definir una plantilla aparte para las vistas de sesiones.

4) Prueba

Dirígete al navegador y agrega a la URL base de tu sistema **/login**. Realiza todo el proceso de login y deberías ver nuestro “*dashboard*” emulado con el mensaje “*¡Bienvenido! Has iniciado sesión correctamente.*”

Una última comprobación a realizar es abrir las herramientas de desarrollo en el navegador y en la zona de almacenamiento comprobar que tenemos las cookies de sesión creadas.



Ahora, cada vez que visitemos cualquier página del sistema, Laravel utilizará la cookie **laravel_session** para obtener la información del usuario que ha iniciado sesión actualmente en el sistema y en el navegador actual.

Estructura de usuarios y sesiones en Laravel

Por defecto Laravel utiliza una estructura de tablas para los usuarios y sus respectivas sesiones, las cuales vienen predefinidas en las migraciones del sistema.

Tabla users

La tabla users es la base de datos central para el sistema de autenticación de Laravel. Por defecto, esta tabla se crea durante las migraciones de la aplicación y contiene los campos esenciales para almacenar la información de los usuarios.

- **id:** Un identificador único de tipo bigint que sirve como clave primaria.
- **name:** Un campo de tipo string para el nombre del usuario.
- **email:** Un campo de tipo string para el correo electrónico del usuario. Suele tener un índice unique para asegurar que no haya dos usuarios con el mismo correo.
- **email_verified_at:** Un campo de tipo timestamp que puede ser nulo, usado para marcar la fecha y hora en que se verificó la dirección de correo electrónico del usuario.
- **password:** Un campo de tipo string donde se almacena la **contraseña encriptada** del usuario. Es crucial que nunca se guarde la contraseña en texto plano, y Laravel se encarga de esto automáticamente usando el hash bcrypt.
- **remember_token:** Un campo de tipo string de 100 caracteres que puede ser nulo, utilizado para la funcionalidad de "recordarme" (remember me). Laravel usa este token para mantener la sesión del usuario a través de múltiples sesiones del navegador.

- **created_at** y **updated_at**: Campos de tipo timestamp que Laravel gestiona automáticamente para registrar la fecha de creación y la última actualización del registro.

Tabla sessions

La tabla sessions es un almacén opcional pero muy común para los datos de la sesión. Cuando el controlador de sesiones se configura para usar la base de datos (configurado en **config/session.php**), Laravel guarda aquí los datos de la sesión en lugar de usar archivos o una caché.

- **id**: Un campo de tipo string que almacena el identificador único de la sesión (Session ID), que también se envía al navegador a través de una cookie.
- **user_id**: Un campo de tipo unsigned bigint que puede ser nulo. Es una clave foránea que enlaza el identificador de la sesión con el ID del usuario en la tabla users si el usuario ha iniciado sesión.
- **ip_address**: Un campo de tipo string que registra la dirección IP desde donde se originó la sesión.
- **user_agent**: Un campo de tipo text que almacena el "User-Agent" del navegador, útil para identificar el dispositivo y el navegador.
- **payload**: Un campo de tipo longtext que guarda los **datos serializados** de la sesión. Aquí es donde Laravel almacena cualquier dato que hayas guardado en la sesión, como mensajes de flash, información de carritos de compra, o cualquier otra variable que necesites persistir entre peticiones.
- **last_activity**: Un campo de tipo integer que almacena una marca de tiempo de la última actividad en la sesión, utilizada para la gestión del tiempo de vida de la sesión.

Modelo User

El modelo User (**app/Models/User.php**) es una representación de la tabla users en la capa de la aplicación. Este modelo extiende la clase Authenticatable, lo que permite gestionar la autenticación sin que tengas que programar la lógica desde cero.

Advertencia

Puede resultar tentador editar el nombre de la tabla de users a usuarios y el modelo de User a Usuario. Pero dado que este modelo conforma gran parte del sistema de autenticación de Laravel, se sugiere fuertemente no modificarlo a menos que estemos dispuestos a modificar parte del funcionamiento interno de Laravel. Si necesitas gestionar usuarios en tu aplicación, utiliza la tabla y el modelo como vienen predefinidos, sin modificar su estructura o funcionamiento.

Tips de seguridad extras

Siempre que trabajemos con autenticación y sesiones (especialmente en PHP) debemos tener en cuenta los siguientes tips:

- Las contraseñas nunca deben guardarse como texto plano.

- Usar algoritmos de encriptación seguros: Argon2, Bcrypt, Scrypt, PBKDF2.
- Evitar algoritmos inseguros para contraseñas: MD5, SHA256, AES128.
- Limitar la cantidad de intentos de inicio de sesión por minuto/hora (rate limiting).
- No almacenar datos sensibles en la sesión del sistema, como la contraseña.
- Sugerencia: no implementar algoritmos propios o un sistema de autenticación personalizado.

Directivas de autenticación

Ahora que contamos con la posibilidad de autenticar usuarios en nuestra aplicación, podemos hacer uso de las diversas directivas que nos permiten condicionar acciones en el sistema dependiendo de si un usuario ha iniciado sesión (es decir, es un usuario registrado) o no (es una visita).

Directivas de vistas

- En las vistas podemos usar la directiva **@auth** para comprobar si hay un usuario autenticado:

@auth

```
<p>Bienvenido, usuario!</p>
```

@endauth

- O **@guest** para comprobar si no existe un usuario autenticado:

@guest

```
<p>Bienvenido, visita.</p>
```

@endguest

Directivas de lógica/controladores

- En los controladores podemos usar **Auth::check()** para comprobar si hay un usuario autenticado:

```
if (Auth::check()) {
    // Hay un usuario autenticado
}
```

- O **Auth::guest()** para comprobar si no existe un usuario autenticado:

```
if (Auth::guest()) {
    // Hay una visita
}
```

```
}
```

Lectura de datos del usuario

También podemos obtener en cualquier momento los datos del usuario actual de la aplicación.

Obtener el usuario completo

- Usando la función ***auth()***

```
$usuario = auth()->user();
```

- Usando la clase ***Auth***

```
$usuario = Auth::user();
```

- Usando ***\$request*** (solamente si estamos en un controlador o recibiendo una petición)

```
$usuario = request()->user();
```

Obtener el ID del usuario

- Usando la clase ***Auth***

```
$id = Auth::id();
```

- Usando la función ***auth()***

```
$id = auth()->id();
```

Alternativa: usar un kit de inicio

A partir de Laravel 12 contamos con nuevos kits de inicio que crean todas las páginas, rutas y lógica necesaria para registrar usuarios, iniciar sesión e incluso un dashboard personalizado.

Estos kits de inicio están codificados utilizando uno de los siguientes 3 frameworks frontends, por lo cual si los elegimos será crucial conocerlos completamente para poder realizar las modificaciones necesarias:

- React
- Vue
- Livewire

Para usarlos, simplemente creamos un nuevo proyecto y seleccionamos el kit de inicio deseado:


```
➔ 7s ○ laravel new nuevo-proyecto
```

```
Which starter kit would you like to install? [None]:
```

```
[none      ] None
[react    ] React
[vue      ] Vue
[livewire] Livewire
```

Luego indicamos si el sistema de autenticación usará Laravel por defecto u otra librería llamada WorkOS (se prefiere Laravel)

```
Which authentication provider do you prefer? [Laravel's built-in authentication]:
[laravel] Laravel's built-in authentication
[workos ] WorkOS (Requires WorkOS account)
```

Y continuamos con la instalación. Al finalizar la misma contaremos con un sistema de login, registro y sesiones con la tecnología seleccionada.

Advertencia

A menos que conozcamos alguna de las 3 herramientas mencionadas previamente, se sugiere no instalar ningún kit de inicio y crear el sistema de login con los pasos descritos previamente.